Parallel Computing Assignment 1

Implementation and experimental report | Measurements: 24 October 2026
Repository: GitGud031005/parallel-computing-assignment

Scope and deliverables

The implementation covers all three required shared-memory parts in C++20: TAS/TTAS/MCS mutex locks, a reusable Sense-Reversing Barrier, and concurrent counters using mutexes, compare-and-swap (CAS), and fetch-and-add (FAA). Standard mutex/barrier implementations provide comparison baselines. TTAS/CAS exponential backoff are additional experiments. Optional MPI-3 one-sided RMA is not implemented.

Requirement	Implementation / evidence
Locks and prime workload	TAS, TTAS, MCS, std::mutex, TTAS-backoff; identical naive is_prime and outside-work contract.
Reusable barrier	Sense reversal and std::barrier; global episodes per second without useful work between episodes.
Counters	Four mutex counters, CAS, FAA, CAS-backoff; increments return the previous value.
Correctness and scaling	Directed tests, SC reference variants, stress runs, raw records, median/IQR plots and reproducible scripts.

All measurements were taken on the submitter's own laptop: 13th Gen Intel(R) Core(TM) i7-13620H (10 physical cores, 16 logical CPUs, 15.7 GiB RAM); 6 performance cores with Hyper-Threading plus 4 efficiency cores; Windows 11 chooses core placement (no affinity pinning); Lenovo Legion Balance power mode on AC power. This is the only shared-memory platform used in the study.

Run: run-laptop-02. Revision: 3d2490e+working-tree. Kernel/build SHA-256: 38a67327a7a796b185cbda8610e235bfac08c2ab067e2ebacf906cfeffe9a8b2

1. Memory model and mutex locks

Cache coherence alone does not establish visibility across multiple variables. A C++ data race on ordinary shared data causes undefined behavior. A lock must establish both mutual exclusion and publication of the previous owner's payload. Unlock uses release semantics; the successful acquiring operation receives the corresponding publication. Relaxed polling is only a waiting optimization [2].

Lock	Mechanism and ordering
TAS	Repeated atomic_flag test_and_set(acquire); clear(release). Repeated central RMW traffic; no FIFO guarantee.
TTAS	Poll with relaxed loads while busy, then exchange(acquire); store(release) on unlock. Reduces RMW traffic during waiting, with contention still possible at release.
MCS	Stable per-worker queue nodes, atomic tail and local waiting flag. FIFO follows tail-exchange order. A preempted owner can block the queue.
std::mutex	Standard-library baseline using the same workload and timing harness. Its internal implementation need not be a spinlock.

MCS initializes its node before tail.exchange(acq_rel). An enqueued worker publishes itself through predecessor.next with a release store and waits using acquire loads. Its predecessor releases the payload by storing waiting=false with release semantics. Node/context lifetime extends through unlock; a context is not copied or moved while in use.

There are two publication paths. With a successor, the waiting flag performs the handoff. With an apparently empty queue, the owner uses strong CAS to reset tail from its own node to null with release semantics. The next acquiring tail exchange receives that publication.

If CAS fails, a successor may have exchanged tail without linking next yet. The owner must wait for that link and then hand off. Weak CAS could fail spuriously when no successor exists and cause an indefinite wait here. A separate expected pointer prevents CAS from overwriting the identity of the node being unlocked. A directed test holds the successor exactly in this enqueue gap.

Spinlocks are blocking synchronization: progress can depend on a particular owner running. FIFO enqueue order is not invocation timestamp order, and a passing stress run does not prove starvation freedom.

2. Sense reversal and concurrent counters

The barrier has a fixed positive participant count P, remaining=P and global sense=false. Each worker has a separate local sense for each barrier instance. An arrival flips local sense and performs remaining.fetch_sub(1, acq_rel). The last arrival resets remaining=P with a relaxed store, then publishes global sense with release. Other participants wait with acquire loads.

The acq_rel RMW chain accumulates the writes preceding each arrival. The last thread receives this chain and publishes it through sense; waiting threads acquire that publication. Reset is sequenced before the release of sense. A second generation cannot finish until every participant has entered it, preventing a delayed participant from missing its previous generation.

One thread_local boolean shared by all barrier instances would be incorrect when instance phase counts differ. Explicit per-worker/per-instance contexts avoid this. Visibility tests use a second barrier before the next round of writes so that the test itself has no data race. The baseline calls std::barrier::arrive_and_wait [3].

Counter	Linearization and ordering
Mutex TAS/TTAS/MCS/std	An ordinary uint64_t increment while holding the lock. Concurrent value() calls acquire the same lock.
CAS	A successful CAS changing x to x+1. Weak-CAS retries recompute desired from the updated expected value.
FAA	fetch_add(1) is one atomic RMW and returns the previous value. Separate load/store would lose updates.
CAS-backoff	The same CAS algorithm with deterministic 1..1024 pause steps after failure, reset after each operation.

CAS/FAA use relaxed ordering because the counter represents only a number; it does not publish other payload. Increment returns the old value. Validated total work prevents uint64_t wrap. Actual atomic lock-freedom is platform dependent; CAS retry is not a per-thread wait-free guarantee. Returned values [0,K) and final K are empirical oracles; linearization points supply the general operation argument.

3. Correctness and course alignment

Test group	Oracle and directed cases
Workload	Naive prime against an independent square-root oracle for N=0..10009; quotient/remainder work distribution including K<P; input overflow.
Locks	Non-overlapping occupancy; protected counts; a separate plain-payload visibility test without telemetry RMW that could hide ordering errors; RAII and multiple instances.
MCS	Controlled enqueue gap and FIFO order, empty-queue handoff and node reuse.
Barrier	P=1/2/3/4/5/8, five seeds, many phases, A-A-B across instances, visibility and a delayed final participant; reject P=0.
Counters	No lost updates, exact old-value range, concurrent reads respecting each thread's completed increments, post-join precedence and no-wrap boundaries.
CLI / analysis	Prime/composite checksums, non-divisible work, invalid inputs; known-answer throughput, IQR/speedup and corrupted-row rejection.

Check run for this report	Result
debug	passed (Debug, sanitizer=none)
release	passed (Release, sanitizer=none)
stress_locks	passed
stress_barriers	passed
stress_counters	passed

All six CTest groups use runtime checks that stay active in Release. SC reference implementations supplement the acquire/release variants; stress increases work tenfold and includes bounded oversubscription. Test timeouts detect a failed run, not correctness. Finite tests and sanitizers do not prove every interleaving.

Sanitizers cannot run in the Windows MinGW-w64 toolchain, so the four C++ test programs were also compiled directly with g++ 13.3 under WSL2 Ubuntu 24.04 on the same laptop (outside CMake, -O1 -g). ASan+UBSan: all four passed with no report. TSan initially aborted with 'unexpected memory mapping' before running any test, a known incompatibility between GCC 13 TSan and newer kernel address layouts; it was rerun with address randomization disabled (setarch -R). TSan then passed all four programs (31 test cases, including the MCS enqueue-gap, barrier visibility and counter uniqueness tests) with zero data-race warnings. Logs are kept in validation-local/wsl-sanitizers.

The algorithms follow course material: TAS/TTAS/MCS in Ch03 pp.17-28, sense reversal in Ch04 pp.8-15, CAS/FAA in Ch02a pp.16-21, and C++ ordering in Ch02b. Spinlock progress and counter specifications follow Ch03/Ch05.

4. Platform and measurement protocol

Property	Recorded value
Platform / OS	laptop-i7-13620H / Windows-11-10.0.26200-SP0
CPU	13th Gen Intel(R) Core(TM) i7-13620H (10 physical cores, 16 logical CPUs, 15.7 GiB RAM); 6 performance cores with Hyper-Threading plus 4 efficiency cores; Windows 11 chooses core placement (no affinity pinning); Lenovo Legion Balance power mode on AC power
Toolchain	g++.exe (Rev3, Built by MSYS2 project) 13.2.0; Release -O3 -DNDEBUG; no LTO or sanitizer.
Atomics / waiting	uint64/bool/pointer lock-free: True/True/True; spin hint x86_pause; 64-byte layout alignment.
Control	none; inherits process affinity; Power Scheme GUID: 85d583c5-cf2e-4197-80fd-3789a227a72c (Legion Balance Mode); AC power

Naive is_prime tries divisors 2..N-1 and returns early on a divisor. Each lock iteration acquires, calls it once, releases, then performs W outside calls. The function is in a separate translation unit with IPO/LTO disabled, and local checksums are consumed after join, so the compiler cannot remove the work. volatile is not used as synchronization.

Program time starts immediately before thread spawn and ends after join, including the independent latch start gate. Context/work allocations precede this timer. Worker interval is max(end)-min(start) after the gate. The barrier executes E episodes with no useful work between; throughput is E/worker_seconds, not P*E/seconds.

The matrix has 711 configurations, 2 warmups and 7 measured repetitions each: 6399 raw samples, 0 failed measured samples and 1 failed warmup(s), excluded from statistics by design: mcs P=14 per_thread W=4 (timeout: process exceeded configured timeout). Each sample is a separate process with a 60 s timeout; configuration order uses fixed shuffle seed 20261016. Median and inclusive IQR are descriptive statistics, not confidence intervals.

Threads P=1/2/3/4/6/8/10/12/14 on 10 physical cores and 16 logical CPUs; P=12/14 use Hyper-Threading siblings. P values that fill every logical CPU are measured in a separate saturation run (sections 5 and 7). Core locks use N=997, W=0/1/2/4/8; fixed-total K=20000 or per-thread I=5000. Sensitivity uses N=97/K=200000 and N=10007/K=2000 with W=0/4. Barriers use E=20000; counters K=1000000. Short runs remain sensitive to startup and scheduling. Per-thread ratios are not strong-scaling speedups.

5. Mutex lock results

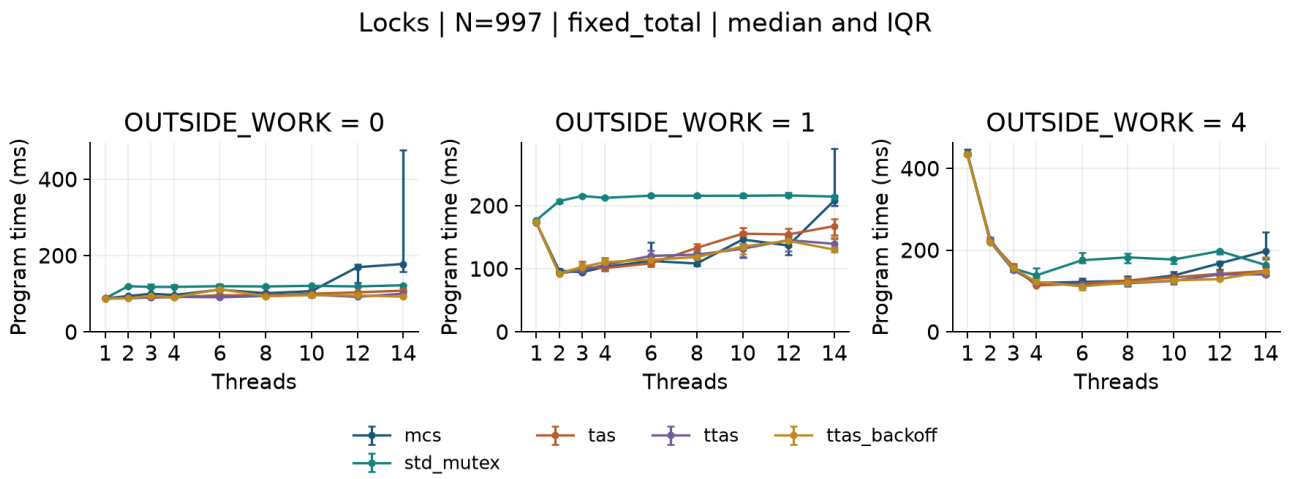


Figure 1. Fixed total K=20000, N=997; median and IQR. Each comparison holds K, N and W constant.

P=14, W=0	Median ms	IQR ms	S(P)
mcs	177.630	157.369-476.455	0.493
std_mutex	121.913	120.205-123.539	0.718
tas	107.687	106.281-108.564	0.813
ttas	99.827	93.370-113.008	0.874
ttas_backoff	92.073	91.751-105.524	0.942

ttas_backoff has the lowest median in this configuration (92.073 ms). This is a workload/platform-specific observation, not a universal ranking. S(P)=T(1)/T(P) uses program time and the same implementation, K, N and W. Small percentage differences alone do not establish statistical significance.

W	Calls in CS	Fastest at P=14	Median ms	Best S(14)	S(P)
0	1/1	ttas_backoff	92.073	ttas_backoff	0.942
1	1/2	ttas_backoff	130.118	ttas_backoff	1.338
2	1/3	ttas	136.414	ttas	1.902
4	1/5	ttas	140.260	ttas	3.098
8	1/9	ttas	142.780	ttas	5.449

With W=0 every prime call is inside the lock, so no speedup is possible: T(1) is about 87 ms and the best any lock can do is stay flat. TAS/TTAS stay within 87-108 ms up to P=14, while std::mutex jumps to about 119 ms as soon as P=2 and then stays flat, and MCS climbs to 169-178 ms at P=12-14.

With W=k outside calls, the critical section still holds 1/(k+1) of the work, so Amdahl's bound limits speedup to k+1. The spinlocks get close at low P: at W=1 they reach about 92 ms at P=2 (1.9x of the 2x bound), but std::mutex is slower than one thread (207-216 ms against 177 ms) for every P from 2 to 14. At W=4 (bound 5x) the best speedup is 3.9x (TTAS-backoff, P=6); at W=8 (bound 9x) it is 5.6x (TAS, P=12).

Past about 6-10 threads the time rises again for every lock. More waiters compete for the same cache line at each release, and this laptop has only 6 performance cores: higher P places threads on Hyper-Threading siblings

and the slower efficiency cores. This placement explanation is a hypothesis consistent with the data, not a measured scheduler trace.

Saturation run (P=12/14/15/16, W=0, 5 measured runs each, 120 s timeout): TAS, TTAS, TTAS-backoff and std::mutex stay at 91-122 ms at every P, but MCS rises from 116 ms (P=12) to 333 ms (P=15) and 2,468 ms (P=16), with single P=16 runs between 0.18 s and 85 s. One warmup run of MCS at P=14 (per-thread, W=4) in the main study also hit the 60 s timeout. MCS hands the lock to one specific successor; when all 16 logical CPUs are busy spinning, that successor can be descheduled, and nobody can enter until Windows runs it again. This is the course's caveat that a preempted thread can block the MCS queue. TAS/TTAS instead let whichever running thread wins the flag enter.

Saturation, ms median [IQR]	P=12	P=14	P=15	P=16
mcs	116 [115-153]	160 [125-173]	333 [216-705]	2,468 [353-61,056]
std_mutex	120 [119-120]	122 [120-125]	121 [121-123]	122 [121-123]
tas	107 [106-111]	113 [105-120]	108 [106-109]	109 [105-110]
ttas	97 [95-104]	98 [92-125]	104 [94-106]	103 [94-123]
ttas_backoff	97 [94-103]	91 [91-93]	117 [98-135]	114 [98-118]

Useful work inside the critical section remains serial. TAS repeatedly updates a shared flag; TTAS reads while busy; MCS replaces central flag contention with queued nodes and handoffs. These are algorithmic explanations, not measured cache-miss attribution.

6. Outside work and critical-section sensitivity

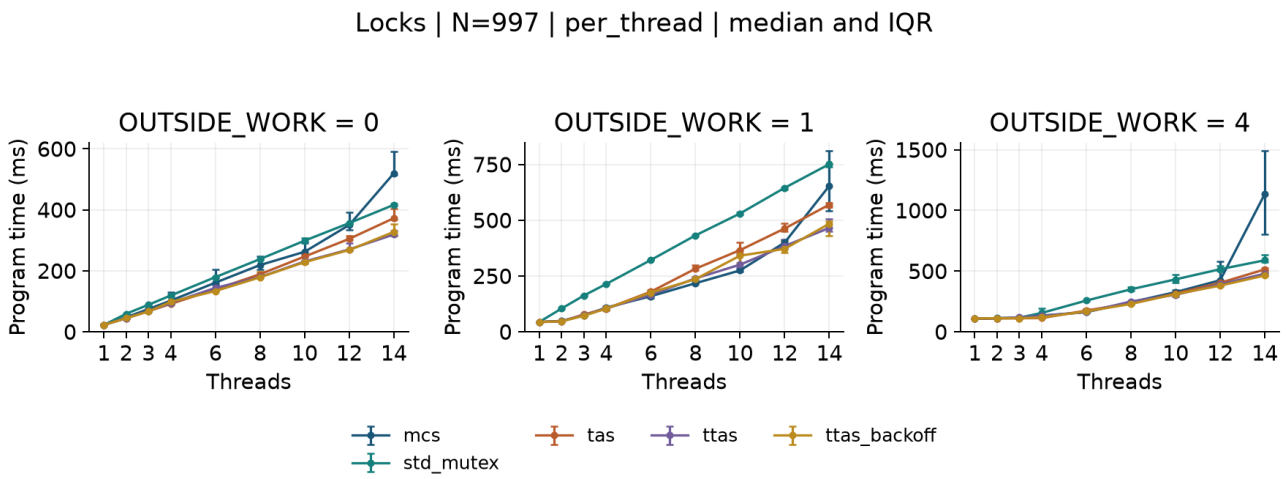


Figure 2. Per-thread I=5000. Total critical-section operations grow as P*I; these curves must not be interpreted as strong scaling.

OUTSIDE_WORK=k puts 1/(k+1) of prime calls inside the critical section. This is a fraction of calls, not elapsed time, because lock/spin/scheduler overhead also contributes. More outside work can reduce contention while increasing total computation. Every sample checks the expected total call checksum.

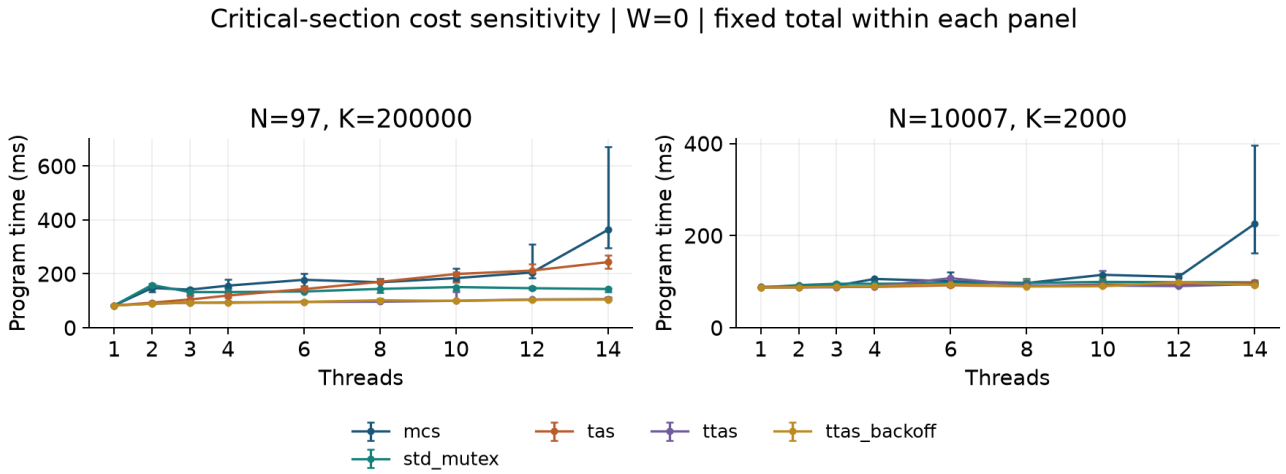


Figure 3. Short/long N sensitivity; K differs between panels to keep durations practical. Compare implementations and P only within a panel.

A short critical section (N=97) means very frequent handoffs. TAS degrades from 82 ms (P=1) to 244 ms (P=14) because every waiter keeps issuing test_and_set on the shared flag, while TTAS stays at 89-106 ms for P=2-14: reading before the RMW is what the slides predict should matter here. With a long critical section (N=10007) handoffs are rare and all locks stay within 87-115 ms except MCS at P=14 (225 ms).

In per-thread mode the total work grows with P, so times grow with P by design. std::mutex is the slowest lock at W=0 from P=2 to P=12, and MCS at P=14, W=4 jumps to 1,135 ms, the same oversubscription effect as above.

Exponential backoff gives TTAS a modest gain where retries are frequent: at N=97, W=4 it lowers the median from 160 to 144 ms at P=4 and from 246 to 226 ms at P=14. Elsewhere TTAS and TTAS-backoff are within run-to-run variation of each other.

Backoff uses 1..1024 pause instructions, not the nanosecond sleeps of the statement's example: a sleep hands the core to the OS scheduler and its minimum granularity is far longer than a short critical section. Backoff can help or hurt depending on retry and waiting costs.

7. Barrier throughput

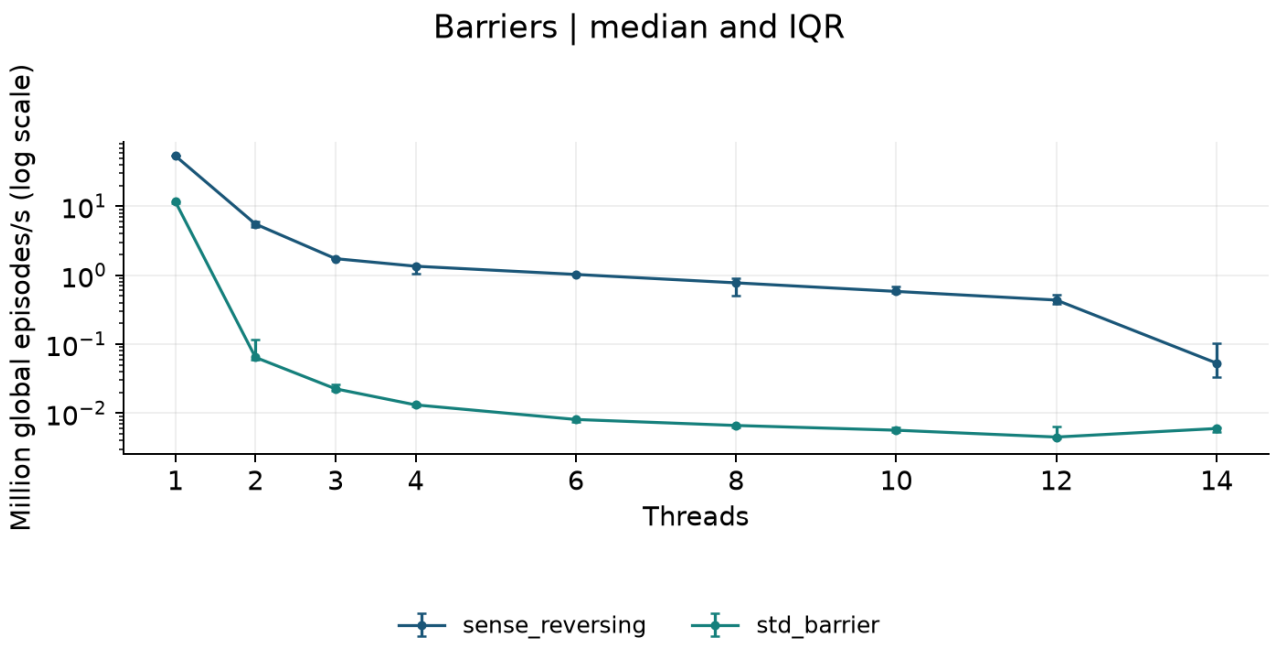


Figure 4. Global episodes per second, median and IQR, without useful work between barriers.

P=14	Median episodes/s	Latency/episode (us)
sense_reversing	53,047.162	18.851
std_barrier	5,974.098	167.389

Each of P workers makes E calls, but the program completes E global episodes. P*E/t measures arrival throughput, a different unit. Timers are read at worker boundaries, not every phase. The standard library and centralized custom barrier have different implementation/platform tradeoffs; the custom remaining/sense state is still shared contention.

The custom barrier is much faster than std::barrier at every P>1: 5.5 million against 64 thousand episodes/s at P=2, and 435 thousand against 4.5 thousand at P=12. A centralized spinning barrier releases waiters with one cache-line update, while the libstdc++ std::barrier is designed to block waiting threads after a short spin, so each episode can pay an OS wake-up; its flat 4.5-6.6 thousand episodes/s from P=8 to P=16 is consistent with a wake-up-bound cost.

At P=1 the 54 million episodes/s only measure call overhead, since no other thread exists. At P=14 the custom barrier drops to 53 thousand episodes/s in the main run and 127 thousand in the saturation run, and to 22 thousand at P=16: one descheduled spinner delays the whole phase. The gap to std::barrier shrinks from about 100x to about 4x, which is the expected trade-off of busy waiting once threads exceed free CPUs.

Saturation, thousand episodes/s	P=12	P=14	P=15	P=16
sense_reversing	426.3 [404.7-597.1]	127.4 [127.2-180.0]	112.6 [106.9-146.4]	22.3 [18.2-37.3]
std_barrier	6.2 [6.2-6.4]	6.1 [6.1-6.2]	6.0 [6.0-6.1]	6.1 [6.0-6.1]

8. Concurrent counter performance

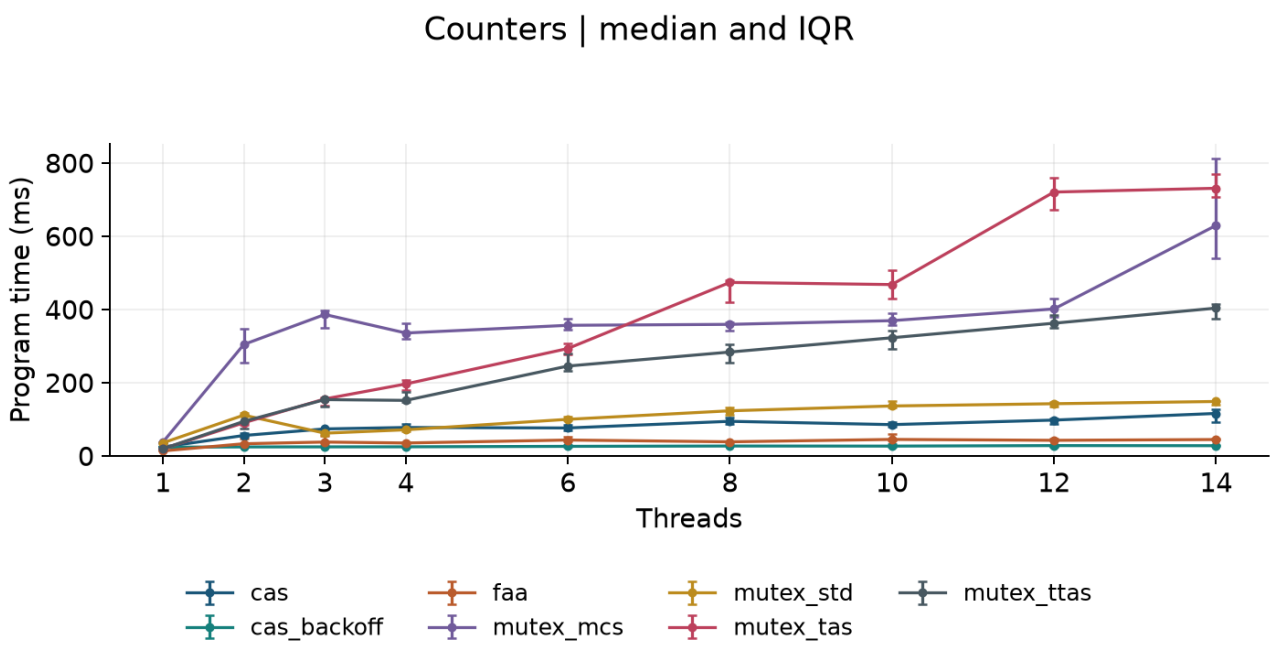


Figure 5. Fixed K=1000000 increments; spawn-to-join program time, median and IQR.

P=14	Median ms	Median increments/s
cas	116.033	8,618,260.197
cas_backoff	27.975	35,746,074.187
faa	44.778	22,332,195.880
mutex_mcs	629.372	1,588,885.681
mutex_std	148.683	6,725,700.381
mutex_tas	730.955	1,368,073.461
mutex_ttas	403.692	2,477,132.966

cas_backoff has the lowest median in this table. All variants validate final K and the old-value checksum. CAS may retry; FAA directly expresses the increment RMW; mutex counters add lock/unlock. All still update one shared numeric location, so more workers do not create independent useful counter work.

No counter becomes faster with more threads: the fastest result overall is FAA with one thread (13.7 ms), because every increment updates one shared cache line and a single thread keeps it in its own cache.

CAS with backoff stays at 24-28 ms for all P, the lowest time from P=2 upward. After a failed CAS the losers pause, so the winner completes many increments in a row while it owns the cache line. This shows throughput, not fairness: backoff concentrates work in one thread at a time.

Plain CAS rises to 116 ms at P=14 because failed attempts waste cache-line transfers, while FAA stays at 33-45 ms since each increment is one RMW that cannot fail. The lock-based counters pay a lock and an unlock per increment: TAS-mutex reaches 731 ms at P=14, and MCS-mutex is already 304 ms at P=2 (37 ms at P=1) because every increment is a handoff to another core.

No diagnostic shared retry counter is added to the timed hot loop. Fast execution does not prove wait-freedom, fairness or starvation freedom.

9. Reproduction, limitations and references

Run from the repository root with CMake >=3.20, a C++20 compiler whose standard library has std::barrier, and Python >=3.10 with the packages in assignments/assignment1/scripts/requirements.txt. Use a fresh output directory; the runner rejects overwriting previous data.

```
cmake -S . -B build/release -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build/release -j 8
ctest --test-dir build/release --output-on-failure
python assignments/assignment1/scripts/run_benchmarks.py \
  --bin-dir build/release/assignments/assignment1 \
  --matrix assignments/assignment1/config/benchmark_matrix.json \
  --platform-id my-computer --output results-local/run-01
```

Then run summarize.py, plot_results.py and create_report.py with the run, summary, figure and checks paths; package_submission.py builds the ZIP. Extract the ZIP into a fresh directory and build/test again.

Limitations: one laptop with a hybrid CPU and no thread pinning, so the OS scheduler and core types add variation; background load was not controlled beyond closing heavy applications. Spin-wait results above 12 threads are dominated by scheduling, so P=15/16 is reported as a separate saturation study. Short runs remain sensitive to startup cost. There are no hardware performance counters, and finite tests and sanitizers do not prove every interleaving. The optional MPI-3 one-sided version was not implemented.

References

[1] Assignment1_v1_16092026.pdf, pp.1-4, supplied course material.

[2] Course slides Ch00-Ch07 (including Ch02a/Ch02b); Ch03 pp.17-28, Ch04 pp.8-15. Page numbers are one-based PDF pages.

[3] C++20 draft N4861: atomics.order, atomics.types.operations, thread.barrier.class. https://timsong-cpp.github.io/cppwp/n4861/

[4] Mellor-Crummey and Scott, Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors, ACM TOCS (1991). https://www.cs.rochester.edu/research/synchronization/pseudocode/ss.html